

SiliconKnights

Theme 2:

Beyond monitoring: AI agents for real-time pod resource discovery and dependency mapping

Team Members:

Soumyadip Das

Shivam Kumar

B Kishan

Aaryan Shyam Pillai



Defining the problem

Kubernetes is now at the edge.

- K3s and MicroK8s run on single-node deployments in factories, smart buildings, and edge data centers — hosting hundreds of pods on constrained, often air-gapped hardware.
- These environments host tens to hundreds of pods across multiple namespaces, each consuming compute, storage, and network resources in patterns that are anything but predictable.
- Existing cloud-native observability tools struggle to capture these hidden cross-resource dependencies in such edge deployments.

Cloud-native tools don't work at the edge

Datadog and Dynatrace require cloud connectivity, are expensive, and are designed for large clusters — not single-node edge deployments.

Service meshes only see the network

Istio and Linkerd are blind to storage contention, CPU starvation, and log queue backpressure — the actual root causes of most edge failures.

Prometheus has no concept of causation

It collects telemetry, but cannot tell you why one pod's behavior is affecting another. It sees numbers — not relationships.

03 Why existing tools fall short

Platform	Cloud Dependency	Manual Configuration	Edge Ready	Resource Correlation
Dynatrace	Yes	Partial	No	Limited
Datadog	Yes	Partial	No	Limited
Istio / Linkerd	No	Automatic	Heavy	Network-only
Prometheus + Grafana Only	No	N/A	Yes	None
(Our Solution)	No	No	Yes	Multi-resource

Existing limitations

- Designed primarily for large cloud-native clusters
- Focused on communication visibility rather than causal resource behavior
- High deployment overhead in constrained edge environments
- No automated tracing of cascading failures

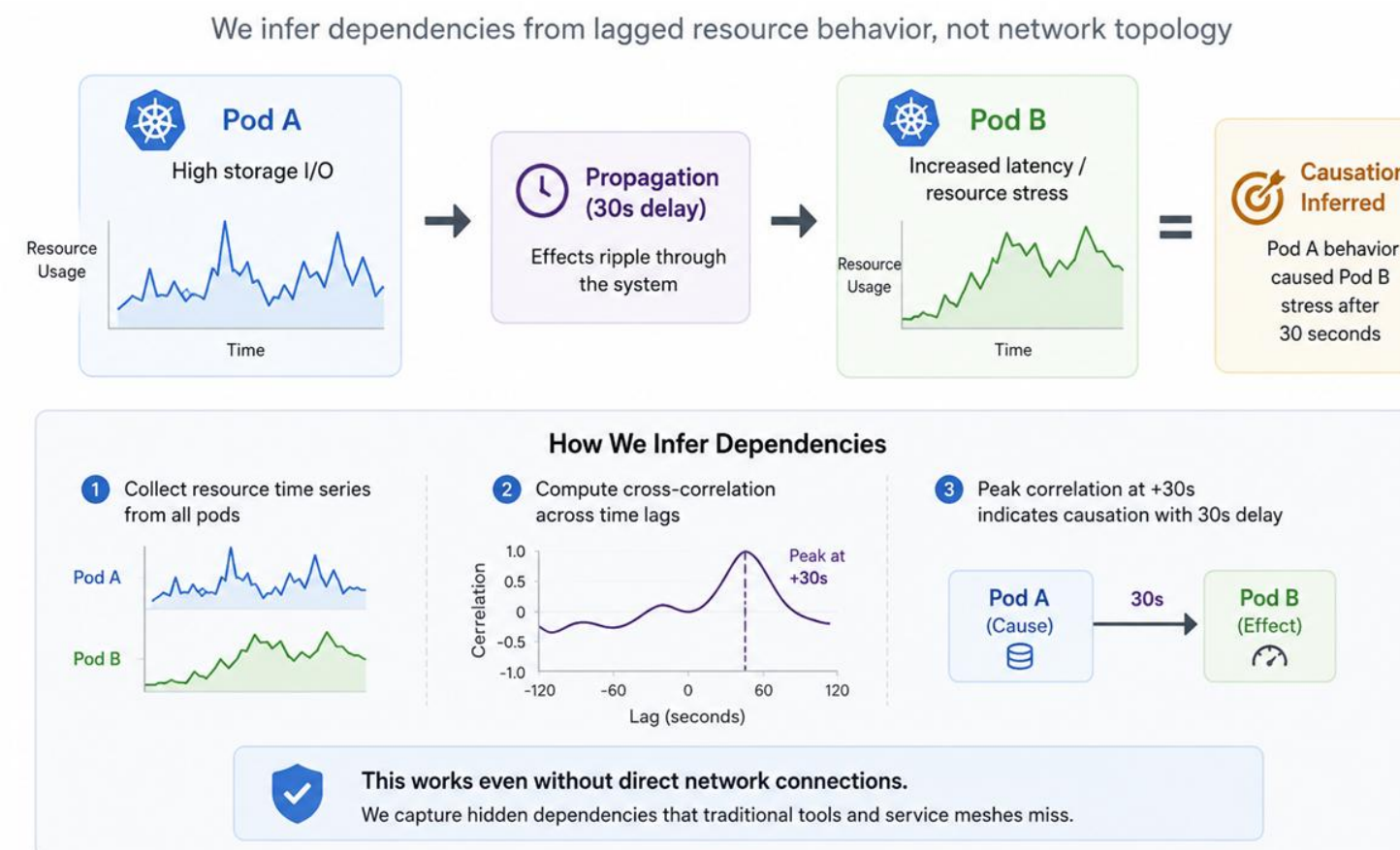
04 Core Insight

Analysis of resource behavior reveals hidden dependencies, identifying:

- infrastructure metrics
- communication paths
- isolated anomalies

However, what they cannot identify are:

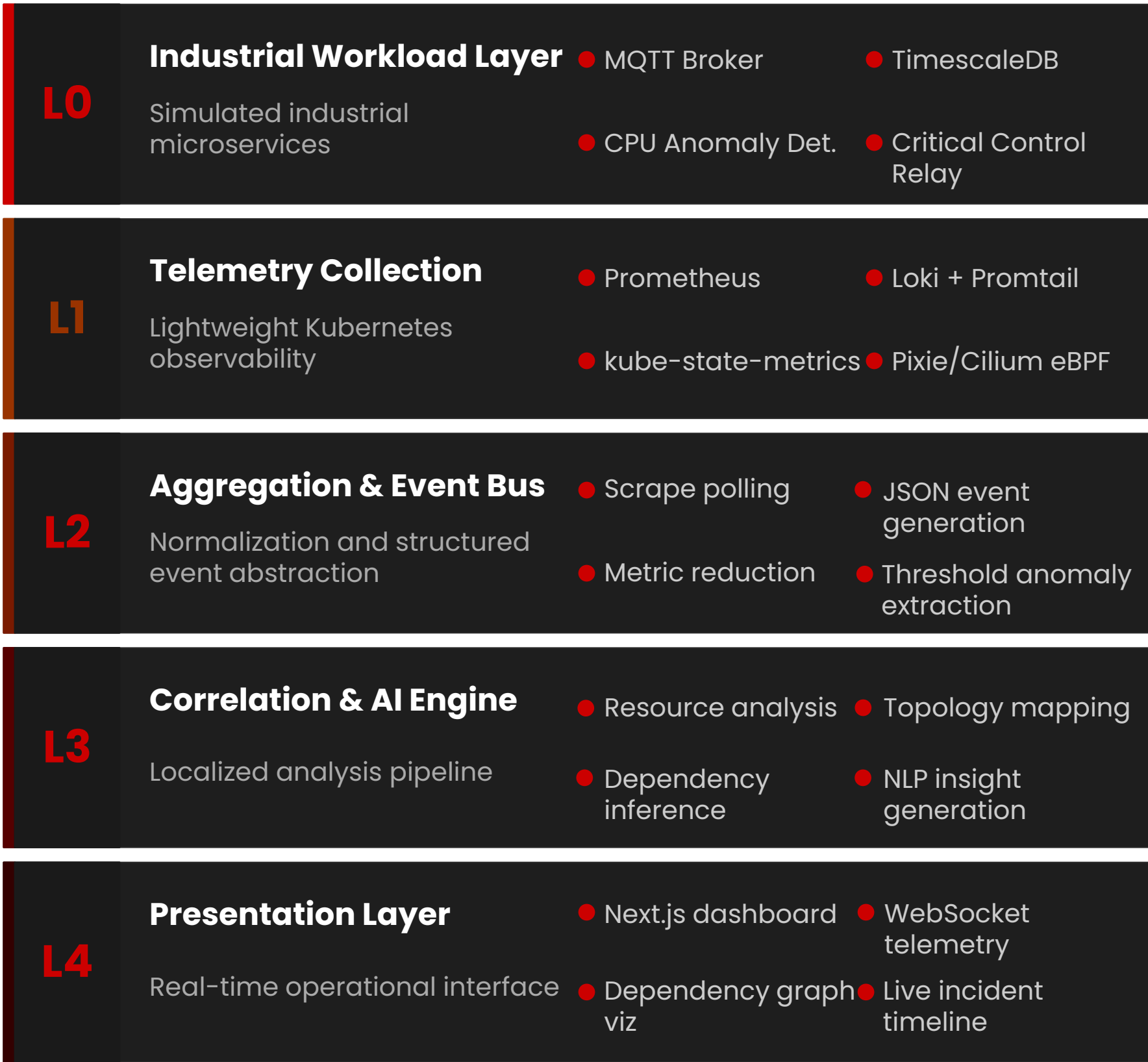
- storage contention cascades
- delayed cross-pod interference
- indirect resource propagation
- causal failure chains



Our solution infers **dependency relationships directly from lagged resource behavior, using time-series correlation analysis.**

05 System Architecture

Layer-by-Layer Architecture



Why simulate the factory floor?

Cannot demonstrate an observability tool without an application to observe. Each pod targets a distinct resource bottleneck; making failure patterns identifiable, reproducible, and attributable.

Why eBPF over code injection?

eBPF discovers inter-pod network dependencies at kernel level; without modifying L0 applications, a critical feature for air-gapped and secure industrial setups.

Why a normalization layer?

Raw Prometheus scrape text fed directly to an LLM will not process. L2 is an architectural requirement; converting noisy distributed telemetry into structured, correlation-ready JSON.

Why local AI agents?

Splitting inference into specialized agents keeps each focused on one task, dramatically improving accuracy. Local Ollama deployment ensures the air-gap.

Why Go backend + Next.js frontend?

Handles high-throughput WebSocket connections and REST calls to the AI container with minimal overhead. React Flow renders live dependency graphs; Recharts renders CPU/PVC time-series in real time.

06 L0: Industrial Workload Layer

Industrial Simulation Environment

MQTT Broker

NETWORK-HEAVY

Chosen to saturate inter-pod network bandwidth — exposes propagation paths to dependent services.

CPU Anomaly Detector

CPU-HEAVY

Chosen to spike CPU utilization — triggers resource starvation cascades detectable by L1 agents.

TimescaleDB

STORAGE-HEAVY

Chosen to generate PVC I/O contention: stresses storage scheduler and exposes latency chains.

Critical Control Relay

LATENCY-SENSITIVE

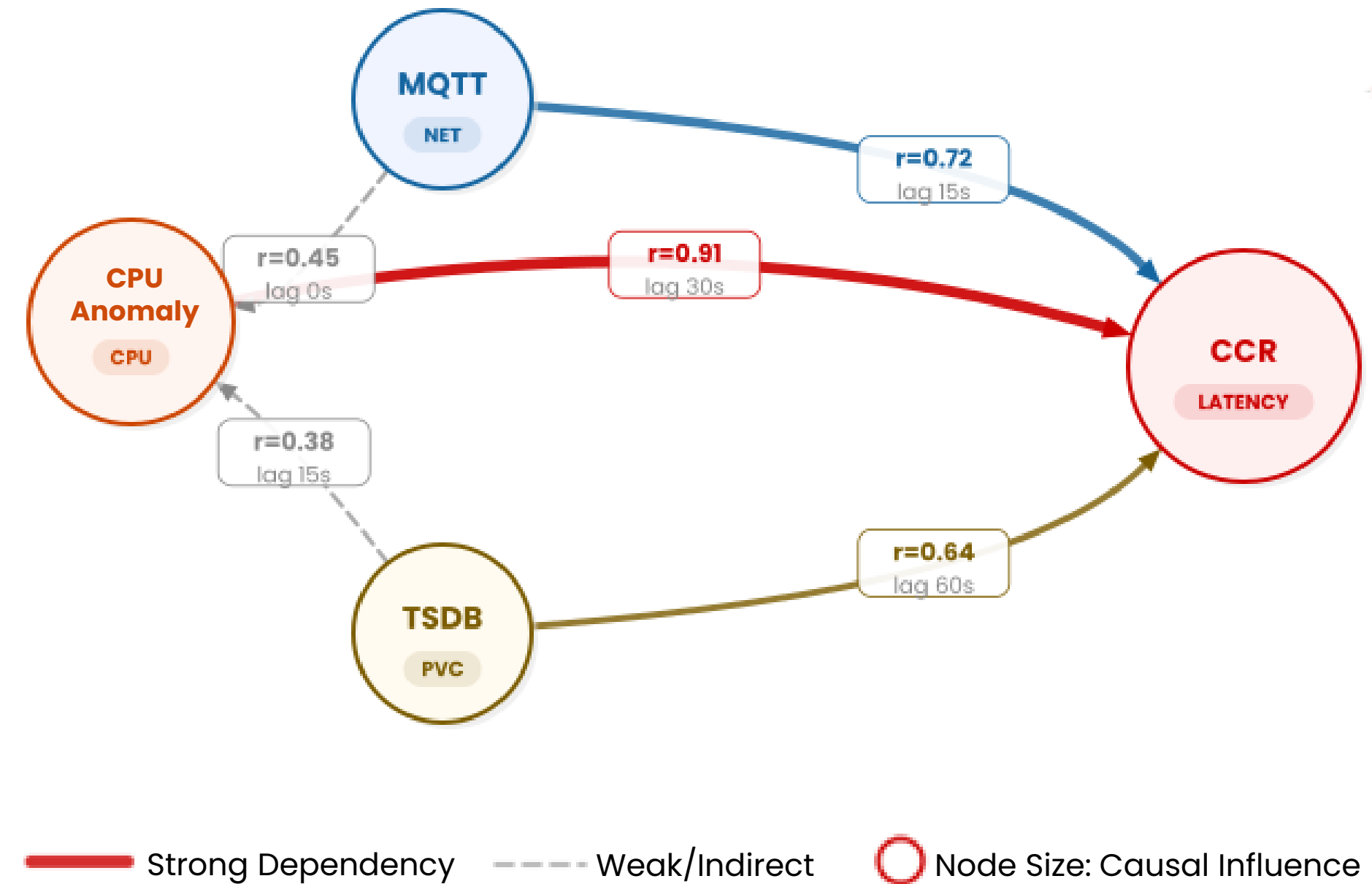
Chosen as the terminal impact node: any upstream failure manifests here as actuation delay.

What this architecture does:

burst CPU PVC contention storage latency network cascade pod interference

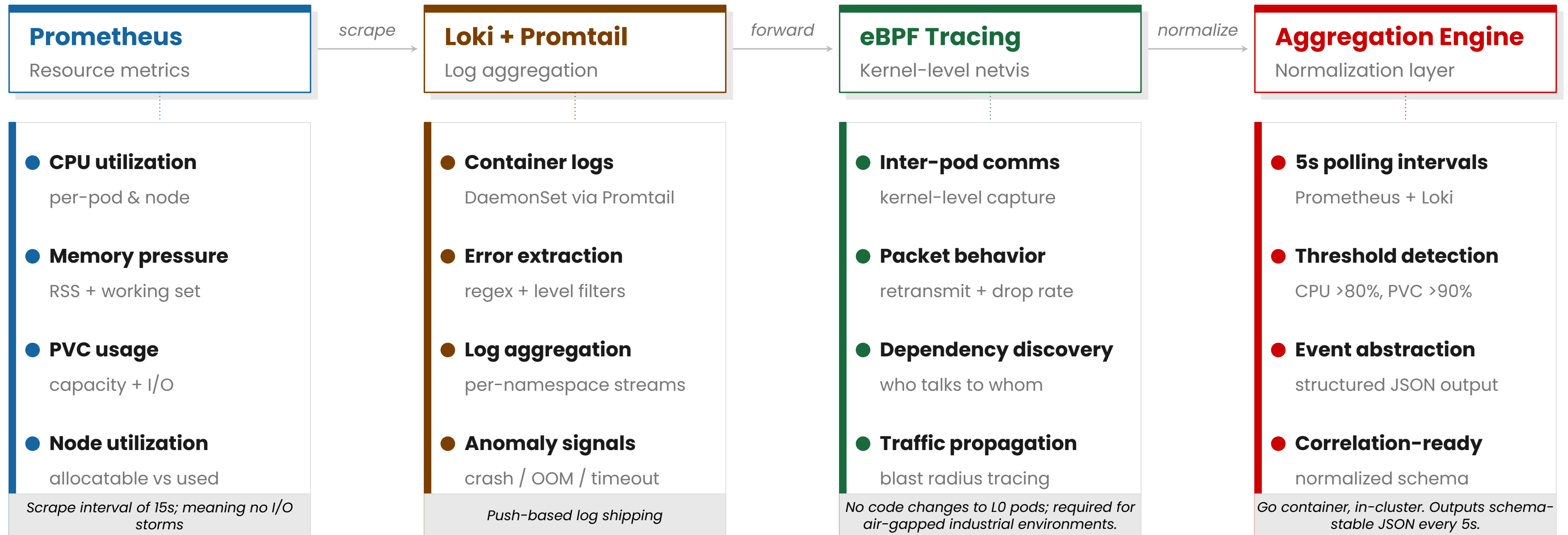
Generates realistic failure conditions for dependency analysis.

Inferred Dependency Graph



07 L1 & L2: Telemetry Collection & Aggregation Bus

Telemetry Collection Pipeline



Why normalization matters

Raw Prometheus scrape text fed directly to an LLM will hallucinate or crash — L2 normalization is a hard architectural requirement, not an optimization.

The aggregation engine polls Prometheus and Loki every 5 seconds, applies threshold rules (CPU >80%, PVC >90%), and emits clean JSON events. This structured output is what makes downstream AI correlation reliable.

08

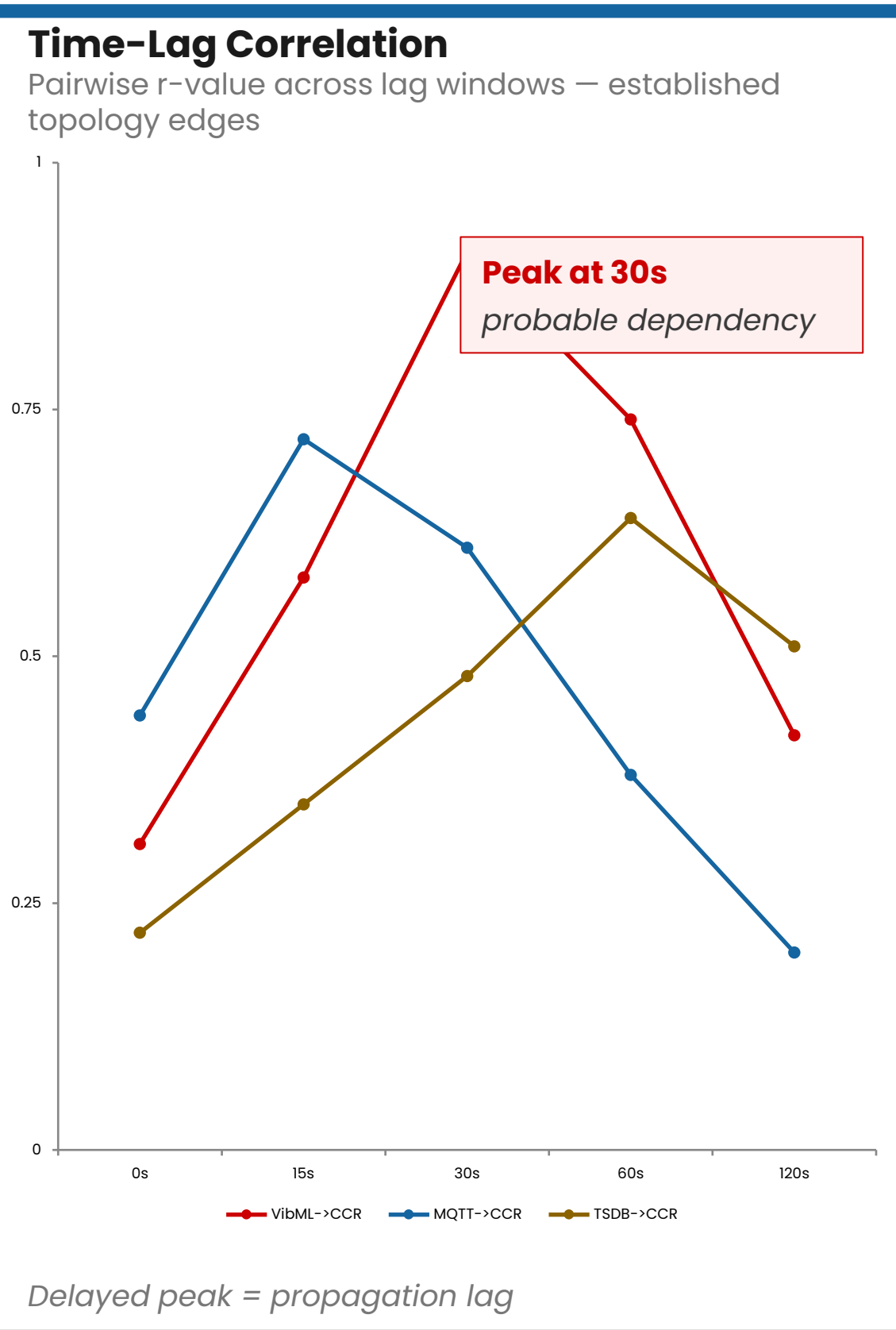
L3: Correlation & Dependency Engine

Resource analysis over an established dependency topology



Live telemetry — Prometheus + kube-state-metrics, 5s intervals

Signal Matrix					
Peak intensity per pod per dimension					
	CPU	MEM	NET	PVC	I/O
MQTT	48%	36%	92%	7%	19%
CPUAD	91%	81%	10%	19%	27%
TSDB	28%	70%	14%	91%	90%
CCR	58%	51%	57%	72%	48%



Signal Ingestion & Correlation Calc

01

Pull normalized JSON
Aggregation engine emits 5s event bundles — CPU, MEM, NET, PVC, I/O per pod.

02

Construct signal vectors
Each pod becomes a time-series matrix. Missing samples interpolated linearly.

03

Pairwise lag correlation
Pearson r computed across all pod pairs at lag windows 0s, 15s, 30s, 60s, 120s.

04

Weight assignment
Peak r-value and its lag written as edge weight onto the known eBPF topology.

Root Cause Ranking + NLP Suggestion

05

Score upstream nodes
Nodes with highest outbound r-values and shortest lag windows ranked as probable root causes.

06

Blast radius mapping
Propagation chain traced via weighted edges: identifying indirect and hidden downstream effects.

07

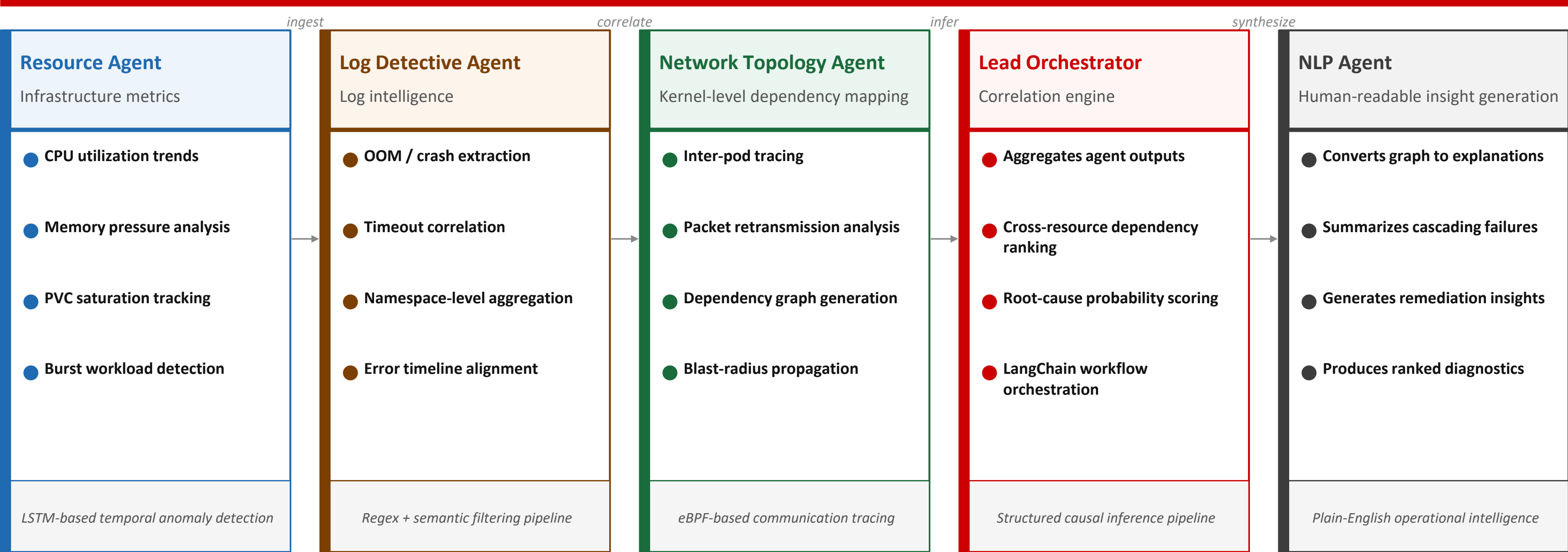
NLP insight generation
Lead Orchestrator passes ranked graph to NLP agent, which translates findings into plain English.

NLP Output
"CPU anomaly spike (t-30s) is the probable root cause of CCR latency degradation. TSDB PVC saturation contributing via storage contention; causing 60s propagation lag."

09

L3: Correlation & Multi AI Engine

Specialized agents for real-time resource correlation and dependency inference



Why multi-agent decomposition matters

Single monolithic LLM pipelines fail under high-volume telemetry streams. The multi-agent architecture isolates resource analysis, log intelligence, network tracing, and causal reasoning into specialized inference layers. This reduces context overload, improves edge inference speed, and enables reliable root-cause ranking in air-gapped industrial environments.

The entire inference pipeline runs locally using Ollama with Llama 3 8B or Phi-3. No cloud APIs or external services are used at any stage of processing. Ensures complete offline operation for secure and air-gapped environments. Reduces latency while maintaining data privacy and deployment independence.

PVC I/O Cascade

Storage contention propagation — detected without manual topology configuration

The Situation

At t=0, the Cooling Monitor begins a large thermal log flush to shared PVC storage. No alert has fired — all pods report healthy. I/O climbs to 94%. The system is watching.

System telemetry at t=0 — I/O storm detected



Key insight

Storage contention identified without service mesh injection, predefined rules, or manual topology setup.

Propagation Chain

Inferred from lag correlation — 5s resolution

Cooling Monitor

I/O: 94% Sequential disk read flush

I/O contention; lag ~0s

Shared PVC Volume

PVC latency +340ms Write queue saturation

write latency; lag 5s

DCIM Bridge

Write deg. 71% Telemetry write stall

log backpressure; lag 15s

Alert Dispatcher

Log delay +8s Log queue backpressure

pod restart; lag 30s

Critical Relay

Response +420ms Actuation latency breach

Engine Analysis

Signal ingestion + correlation output for this scenario

r=0.88 **CoolMon → DCIM**
lag 5s Dependency strength: HIGH

r=0.74 **DCIM → AlertDisp**
lag 15s Dependency strength: HIGH

r=0.61 **AlertDisp → CCR**
lag 30s Dependency strength: MED

r=0.52 **CoolMon → CCR**
lag 45s Dependency strength: MED

System Output

✓ **Storage contention detected**

✓ **Dependency chain inferred**

✓ **Root cause isolated upstream**

✓ **Mitigation recommendation**

NLP Agent

"cooling-monitor I/O spike is the probable root cause. DCIM Bridge write degradation follows with 5s lag. Recommend PVC QoS throttling on cooling-monitor."

01 Correlation False Positives

⚠ Risk: High Pearson r values between unrelated pods sharing periodic background activity.

✓ Mitigation: Require high r across multiple independent lag windows + eBPF network confirmation before inferring strong dependencies.

02 LLM Hallucination on Edge Hardware

⚠ Risk: Quantized local models (Phi-3) can produce incorrect summaries given malformed input.

✓ Mitigation: L2 normalization ensures the model only receives clean structured JSON — never raw Prometheus text.

03 eBPF Kernel Compatibility

⚠ Risk: eBPF requires specific kernel versions; may be unavailable on some edge K3s distributions.

✓ Mitigation: Graceful degradation to network-policy-based topology inference with reduced edge confidence scores.

04 Single-Node Resource Contention

⚠ Risk: Running Prometheus, Loki, eBPF, Go aggregator, and LLM on the same node as workloads.

✓ Mitigation: All observability components in dedicated low-priority namespace with strict CPU/MEM limits; 15s scrape + 5s polling tuned for minimum overhead.

PHASE 01

Foundation

- Deploy K3s cluster locally
- Stand up L0 simulation pods (MQTT, CPUAD, TSDB, CCR)
- Configure Prometheus, Loki, Promtail, eBPF via Helm charts
- Validate raw telemetry on all 5 signal dimensions

PHASE 02

Intelligence

- Build L2 Go aggregation container
- Implement lag correlation engine (Python, NumPy/SciPy)
- Validate dependency inference on known synthetic failure scenarios
- Confirm root cause ranking before any AI integration

PHASE 03

AI Integration

- Integrate LangChain multi-agent pipeline with local Ollama
- Deploy Llama 3 8B or Phi-3 (air-gap preserved)
- Tune agent prompts for reliability on structured JSON input
- Test NLP output quality across multiple incident types

PHASE 04

Dashboard & Demo

- Build Next.js frontend + Go WebSocket backend
- Wire all layers end-to-end (L0 → L1 → L2 → L3 → L4)
- Record live demo of PVC cascade with real-time detection
- Final prototype: < 30s detection, NLP summary, air-gapped

What Round 2 Will Demonstrate

1. Trigger a live PVC I/O cascade on the K3s cluster

All pods report healthy — no alert has fired

2. System detects I/O storm within seconds

Cooling Monitor flagged at 94% I/O — 5 signal dimensions tracked

3. 5-hop propagation chain traced automatically

CoolMon → DCIM → AlertDisp → CCR — with lag at each hop

4. Plain-English NLP summary generated locally

"cooling-monitor I/O spike is the probable root cause..."

5. Live dependency graph updates in real time on dashboard

React Flow renders causal graph; Recharts shows time-series

Prototype Capabilities

- **Fully Air-Gapped**

No cloud calls. Local Ollama + Llama 3. Works offline.

- **Zero-Config Discovery**

eBPF infers topology without modifying workloads.

- **Multi-Resource Cascade**

Storage → App → Log → Control chains detected.

- **< 30s Detection**

From I/O storm trigger to NLP root cause output.

- **Live Dashboard**

WebSocket-driven React Flow graph updates in real time.